

ChatX3 API

Quick Start Guide

The ChatX3 API lets you send a question to the ChatX3 assistant and receive a Markdown-formatted answer specialized for the Sage X3 ecosystem (support, L4G development, documentation retrieval). This guide covers authentication, the request and response format, usage limits, and current restrictions.

Endpoint

```
POST https://akfcgzazfvqipbjvdmn.supabase.co/functions/v1/api-ask
```

All requests use the POST method with a JSON body.

Authentication

Every request must include your personal API key in the x-api-key header.

```
x-api-key: <YOUR_API_KEY>
```

You can find and copy your API key in **User Management -> API Key** (use the *Copy API key* button next to your user). Keep this key private: it identifies you and is tied to your usage limits.

Requests without a valid key are rejected with a **401** response.

Request

Body fields

Field	Required	Type	Description
message_content	Yes	string	The question or message to send to the assistant.

Minimal example

```
{
  "message_content": "How to create a Syracuse user?"
}
```

Example — curl

```
curl -X POST "https://akfcgzazfvqipbjvdmn.supabase.co/functions/v1/api-ask" \
  -H "x-api-key: <YOUR_API_KEY>" \
  -H "Content-Type: application/json" \
  -d '{"message_content": "How to create a Syracuse user?"}'
```

Example — JavaScript (fetch)

```
const res = await fetch(
  "https://akfcgzazfvqipbjvdmn.supabase.co/functions/v1/api-ask",
  {
    method: "POST",
```

```

    headers: {
      "x-api-key": "<YOUR_API_KEY>",
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      message_content: "How to create a Syracuse user?"
    })
  }
);

const data = await res.json();
console.log(data.message);

```

Example — Python (requests)

```

import requests

url = "https://akfcgzazfvqipbjvdmn.supabase.co/functions/v1/api-ask"
headers = {
  "x-api-key": "<YOUR_API_KEY>",
  "Content-Type": "application/json",
}
payload = {"message_content": "How to create a Syracuse user?"}

# Answers can take up to 120 s, so set a generous timeout.
response = requests.post(url, json=payload, headers=headers, timeout=130)
response.raise_for_status()

data = response.json()
if data.get("success"):
  print(data["message"])
  print("conversation_id:", data["conversation_id"])
else:
  print("Error:", data.get("error"))

```

Example — n8n (HTTP Request node)

Configure an HTTP Request node as follows:

Setting	Value
Method	POST
URL	https://akfcgzazfvqipbjvdmn.supabase.co/functions/v1/api-ask
Authentication	None (use a header, see below)
Send Headers	On — add x-api-key = <YOUR_API_KEY> and Content-Type = application/json
Send Body	On — Body Content Type: JSON
Body (JSON)	{ "message_content": "How to create a Syracuse user?" }
Timeout	Set to at least 130000 ms (Options -> Timeout)

The assistant's answer is available downstream as `{{ $json.message }}`, with `{{ $json.conversation_id }}` and `{{ $json.message_id }}` also returned.

Response

Success (200 OK)

```
{
  "success": true,
  "message": "# Creating a Syracuse User in Sage X3 V11\n\nSyracuse users are...",
  "message_id": "63debdac-e0c9-42ae-b51d-d0f904603376",
  "conversation_id": "api_20260624_400330"
}
```

Field	Description
success	true when the request succeeded.
message	The assistant's answer, formatted in Markdown . Render it as Markdown for best readability.
message_id	Unique identifier of the assistant's reply.
conversation_id	Identifier of the conversation. Returned even when you did not provide one.

The answer is in **Markdown** (headings, bold, lists, code blocks). Render it with a Markdown viewer rather than displaying it as raw text.

Usage limits

To keep the service responsive and protect against abuse, requests are limited per user across three rolling windows:

Window	Limit
4 hours	50 messages
7 days	250 messages
30 days	800 messages

When a limit is reached, the API returns a **429** response instead of an answer:

```
{
  "success": false,
  "rate_limited": true,
  "error": "You have reached your limit of 50 messages per 4 hours. Try again in 2 h.",
  "limit": 50,
  "window": "4 hours",
  "retry_at": "2026-06-24T18:30:00.000Z",
  "retry_after_seconds": 7200
}
```

The response also includes a standard `Retry-After` header (in seconds). Your integration should detect 429, read `retry_after_seconds` (or `retry_at`), and wait before retrying rather than sending repeated requests.

Current limitations

The API is being actively developed. As of now, please note the following restrictions:

- **No document handling.** You cannot attach or upload files (PDF, Word, images, etc.). The assistant answers only from the `message_content` text and its own knowledge base. File-based input is planned but not yet available.
- **No conversation memory.** Each request is answered independently. Even though a `conversation_id` is returned, the assistant does not yet use the history of previous messages as context. A follow-up question that relies on an earlier exchange will not work as expected, so restate the full context in each `message_content`.
- **Synchronous processing with a delay.** Answers are generated on demand and typically take 20 to 40 seconds, with a hard maximum of 120 seconds. Set your client timeout to at least 120 seconds. If processing exceeds that limit, the API returns an error and you should retry.

Error responses

Status	Meaning	Typical cause
400	Bad request	Missing <code>message_content</code> .
401	Unauthorized	Missing or invalid API key.
429	Too many requests	A usage limit was reached (see above).
502	Upstream error	The AI service returned an error. Retry.
500	Server error	Unexpected error, including a processing timeout. Retry.

All error responses share the same shape:

```
{
  "success": false,
  "error": "Description of what went wrong."
}
```

Always check the `success` field (and the HTTP status) before reading `message`.

Recommended client behavior

1. Always send the `x-api-key` header.
2. Use a request timeout of at least **120 seconds**.
3. Treat `success === false` and any non-200 status as a failure, and surface `error` to the user.
4. On 429, back off using `retry_after_seconds` before retrying.
5. Render `message` as **Markdown**.
6. Because there is no conversation memory yet, include all relevant context directly in `message_content`.